```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

```python
# from google.colab import drive
# drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```python
path = Path("twitter_training.csv")  # update if needed

# Check if file exists
if not path.exists():
    raise FileNotFoundError(f"File not found: {path}")
df = pd.read_csv(path, header = None)
```

```python
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 74682,\n  \"fields\":
[\n    {\n      \"column\": 0,\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3740,\n        \"min\": 1,\n
\"max\": 13200,\n        \"num_unique_values\": 12447,\n
\"samples\": [\n          1616,\n          2660,\n          2335\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": 1,\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 32,\n
\"samples\": [\n          \"Cyberpunk2077\",\n
\"Microsoft\",\n          \"TomClancysRainbowSix\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": 2,\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"Neutral\",\n          \"Irrelevant\",\n
\"Positive\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": 3,\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 69491,\n        \"samples\": [\n
\"Thanks to @ Kain0025 for the raid. Thanks to @ gamingstreams and @
velonese002 for the bitts! And thanks to @ ColTrysTohete for hanging
out and hanging out!. I hope to continue streaming regularly.. watch
the w / @ Cohtstreams _ coming live!\",\n          \"How not to get
bored about every damn thing in life.\",\n          \"The Best Way to
Protect the Samsung Galaxy Note10+ buff.ly/2zkjIhU <unk> ^\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```

```
df.columns = ['id' , 'entity' , 'sentiment' , 'text']

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74682 entries, 0 to 74681
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   id         74682 non-null  int64
 1   entity     74682 non-null  object
 2   sentiment  74682 non-null  object
 3   text       73996 non-null  object
dtypes: int64(1), object(3)
memory usage: 2.3+ MB
```

#Data Cleaning

```
df.isnull().sum()
```

```
id              0
entity          0
sentiment       0
text          686
dtype: int64
```

```
df = df.dropna(subset = ['text'])

df['sentiment'].value_counts()
```

```
sentiment
Negative      22358
Positive      20655
Neutral       18108
Irrelevant    12875
Name: count, dtype: int64
```

```
df.isnull().sum()
```

```
id            0
entity        0
sentiment     0
text          0
dtype: int64
```

```
duplicate_rows = df.duplicated(keep = False)

if duplicate_rows.any():
    print(df[duplicate_rows])
    # Print the rows that are duplicates
```

```
else:
    print("No duplicates found.")
```

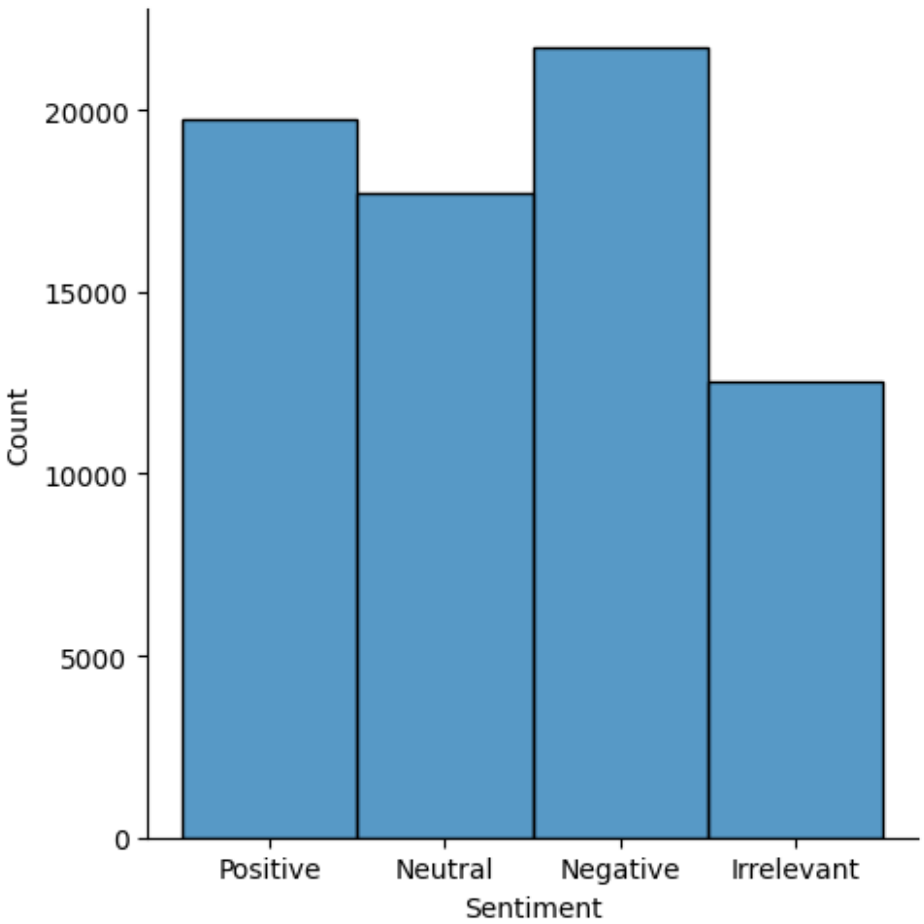
```
         id       entity sentiment  \
18     2404  Borderlands  Positive
20     2404  Borderlands  Positive
25     2405  Borderlands  Negative
26     2405  Borderlands  Negative
48     2409  Borderlands   Neutral
...     ...          ...       ...
74589  9185       Nvidia   Neutral
74616  9190       Nvidia  Positive
74619  9190       Nvidia  Positive
74635  9193       Nvidia  Positive
74636  9193       Nvidia  Positive

                                                    text
18     that was the first borderlands session in a lo...
20     that was the first borderlands session in a lo...
25     The biggest disappointment of my life came a y...
26     The biggest disappointment of my life came a y...
48     Blaming Sight for Tardiness! A little bit of b...
...                                                  ...
74589  Heard people are having issues with ordering t...
74616  This news about the Nvidia 3000 series is ligi...
74619  This news about the Nvidia 3000 series is ligi...
74635                          Love EVERYTHING about it.
74636                          Love EVERYTHING about it.

[4305 rows x 4 columns]
```

```
df.drop_duplicates(keep='first', inplace=True)

duplicate_rows = df.duplicated(keep=False)

# Check if the duplicates have been deleted correctly
if duplicate_rows.any():
    print(df[duplicate_rows])
else:
    print("No duplicates found.")
```

```
No duplicates found.
```

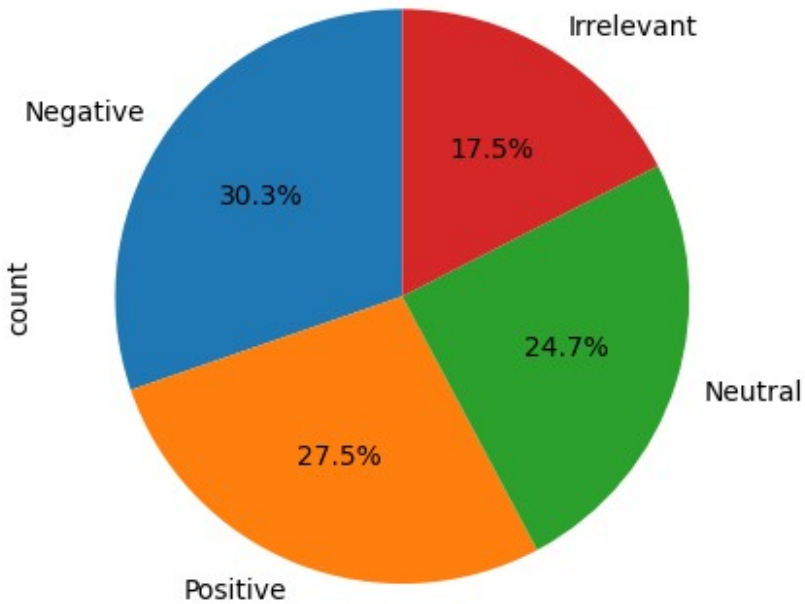
```
sns.displot(df['sentiment']).set(xlabel='Sentiment', ylabel='Count')
```

```
<seaborn.axisgrid.FacetGrid at 0x78444d6d59d0>
```

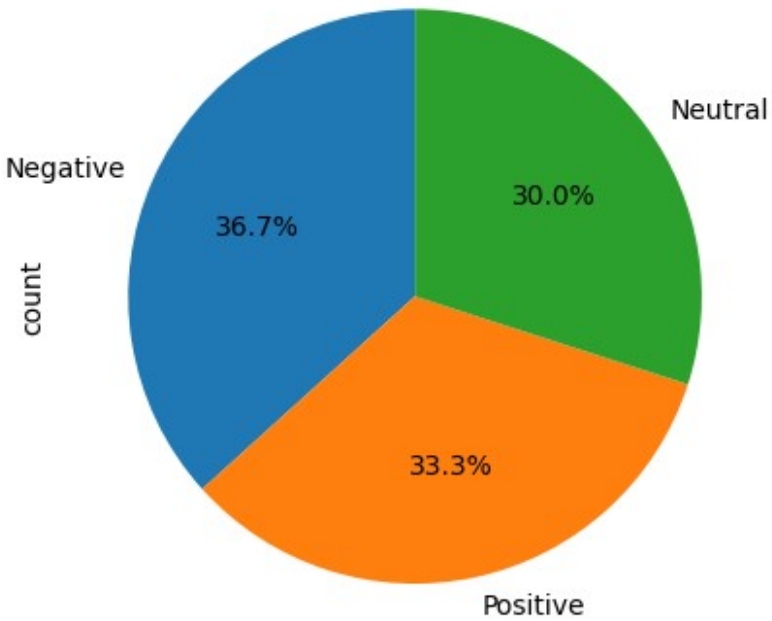



```
df['sentiment'].value_counts().plot.pie( autopct='%1.1f%%',
startangle=90, legend=False)
```

```
<Axes: ylabel='count'>
```




```
df.drop(df[df['sentiment'] == 'Irrelevant'].index, inplace=True)

df['sentiment'].value_counts()

sentiment
Negative    21698
Positive    19713
Neutral     17708
Name: count, dtype: int64

df['sentiment'].value_counts().plot.pie( autopct='%1.1f%%',
startangle=90, legend=False)

<Axes: ylabel='count'>
```



##PreProessing

```
import re
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

stopwords = set(stopwords.words('english'))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
def clean_text(text):
  text = text.lower()
  text = re.sub(r'http\S+' ,'' ,text) #links
  text = re.sub(r'@[^\s]+' ,'' ,text) #mentions
  text = re.sub(r'#[^\s]+' ,'' ,text) #hashtags
  text = re.sub(r'[^a-zA-z]' , ' ' ,text)
  text = text.split()
  text = [word for word in text if word not in stopwords]
  return  ' '.join(text)

df['clean_text'] = df['text'].apply(clean_text)

from sklearn.preprocessing import LabelEncoder
enc = LabelEncoder()
df['sentiment'] = enc.fit_transform(df['sentiment'])
```

```
enc.classes_
```

```
array(['Negative', 'Neutral', 'Positive'], dtype=object)
```

```
label_mapping = dict(zip(enc.classes_, enc.transform(enc.classes_)))
print(label_mapping)
reverse_mapping = dict(zip(enc.transform(enc.classes_), enc.classes_))
print(reverse_mapping)
```

```
{'Negative': np.int64(0), 'Neutral': np.int64(1), 'Positive':
np.int64(2)}
{np.int64(0): 'Negative', np.int64(1): 'Neutral', np.int64(2):
'Positive'}
```

```
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 59119,\n  \"fields\":
[\n    {\n      \"column\": \"id\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3760,\n        \"min\": 1,\n
\"max\": 13198,\n        \"num_unique_values\": 10282,\n
\"samples\": [\n          7348,\n          3883,\n          9312\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"entity\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
32,\n        \"samples\": [\n          \"Cyberpunk2077\",\n
\"Microsoft\",\n          \"TomClancysRainbowSix\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sentiment\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 2,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          2,\n
1,\n          0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"text\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 57297,\n        \"samples\": [\n          \"If
@ Treyarch does not remove the damn shrubs from Cartel, this card is
doomed to fail.\",\n          \"The only pity is that there won't be a
live broadcast tonight, which will have a lot of problems with the
internet and it won't be able to maintain the stream.\",\n
\"@GhostRecon ever gonna fix mercenaries mode in woodlands ? So many
people in PVP talk about it but it only worked for like 2 months.
Would love to see it going again was really fun.\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"clean_text\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 50296,\n        \"samples\": [\n
\"skin\",\n          \"stay mario kart borderlands shit compete
someone else main motherfuckers hate people play gta last us\",\n
\"blatant ea warcraft ripoff goes offline soon blizzard files file
lawsuit pcgamer com blatant world view\"\n        ],\n
```

\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df"}

##Train-Test Split

```
X = df['clean_text']
y = df['sentiment']

X_train , X_test , y_train , y_test = train_test_split(X , y ,
test_size = 0.2 , random_state = 42)
```

##Feature Extraction (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=5000)

X_train = tfidf.fit_transform(X_train)
X_test = tfidf.transform(X_test)

y = df['sentiment']

print(X_train.shape)
print(y_train.shape)
```

```
(47295, 5000)
(47295,)
```

#Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train , y_train)
```

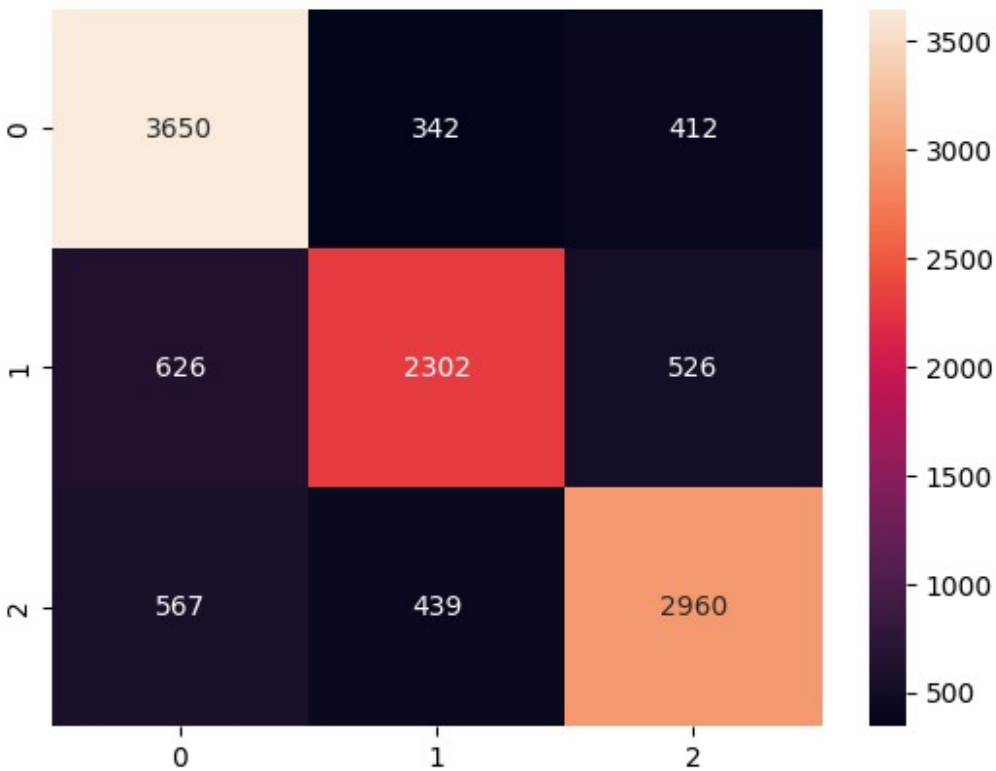
```
LogisticRegression()
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7537212449255751
              precision    recall  f1-score   support

           0       0.75      0.83      0.79      4404
           1       0.75      0.67      0.70      3454
           2       0.76      0.75      0.75      3966

    accuracy                           0.75     11824
   macro avg       0.75      0.75      0.75     11824
```

```
weighted avg       0.75      0.75      0.75     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.show()
```



#Naive Bayes

```
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
model.fit(X_train , y_train)
```

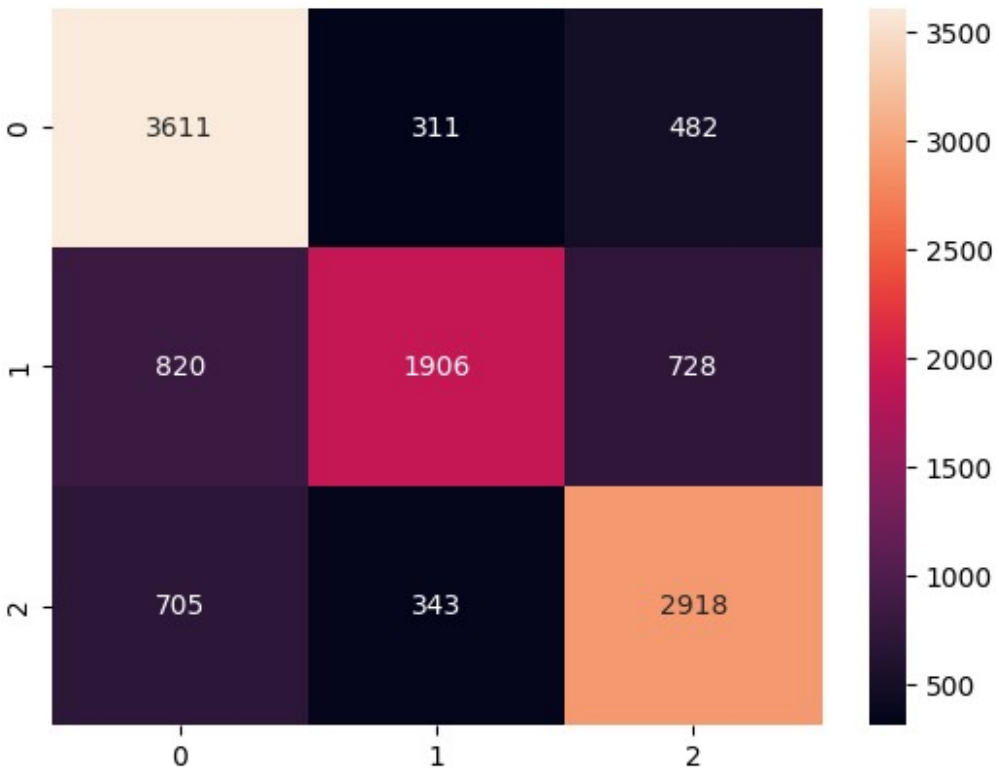
```
MultinomialNB()
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7133795669824087
              precision    recall  f1-score   support

           0       0.70      0.82      0.76      4404
```

```
           1       0.74      0.55      0.63      3454
           2       0.71      0.74      0.72      3966

    accuracy                           0.71     11824
   macro avg       0.72      0.70      0.70     11824
weighted avg       0.72      0.71      0.71     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.show()
```



#SVM

```
from sklearn.svm import LinearSVC

model = LinearSVC()
model.fit(X_train , y_train)
```

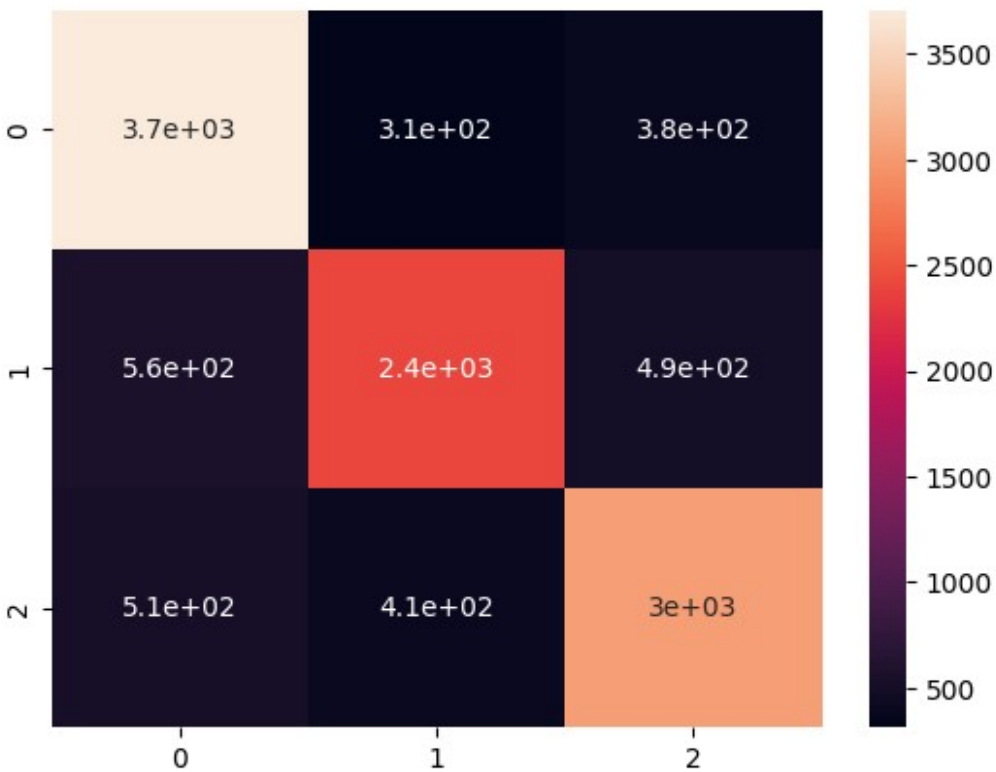
```
LinearSVC()
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7744418132611637
              precision    recall  f1-score   support

           0       0.77      0.84      0.81      4404
           1       0.77      0.70      0.73      3454
           2       0.78      0.77      0.77      3966

    accuracy                           0.77     11824
   macro avg       0.77      0.77      0.77     11824
weighted avg       0.77      0.77      0.77     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.show()
```



#Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

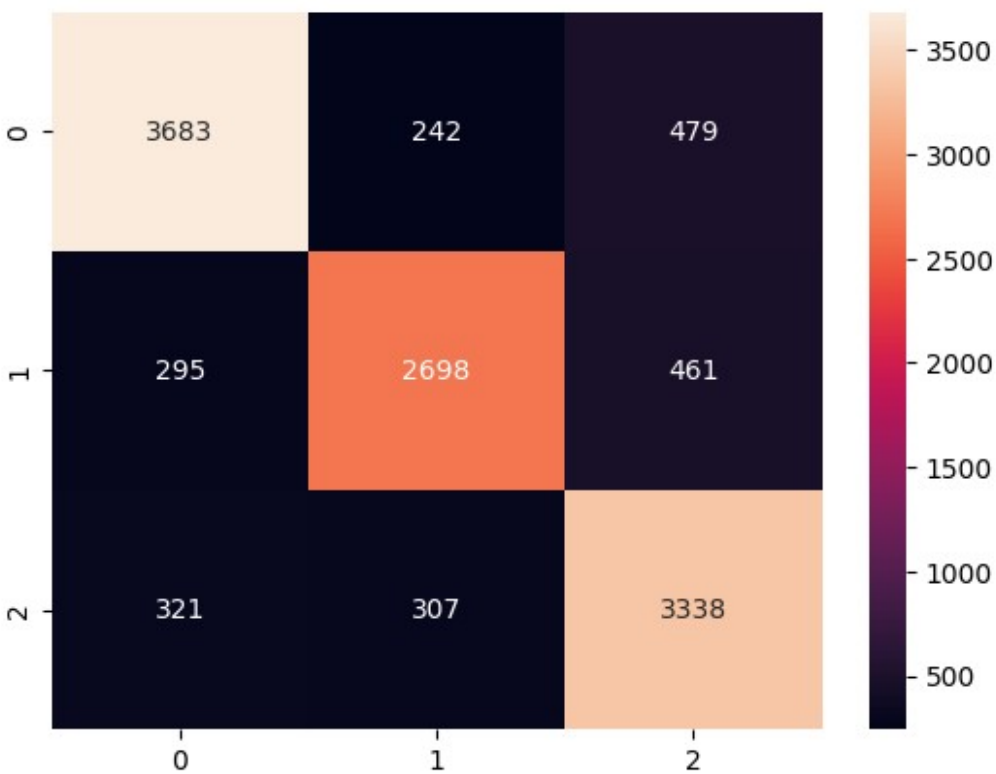
```
model = DecisionTreeClassifier()
model.fit(X_train , y_train)
```

```
DecisionTreeClassifier()
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.8219722598105548
              precision    recall  f1-score   support

           0       0.86      0.84      0.85      4404
           1       0.83      0.78      0.81      3454
           2       0.78      0.84      0.81      3966

    accuracy                           0.82     11824
   macro avg       0.82      0.82      0.82     11824
weighted avg       0.82      0.82      0.82     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.show()
```



#Random Forest

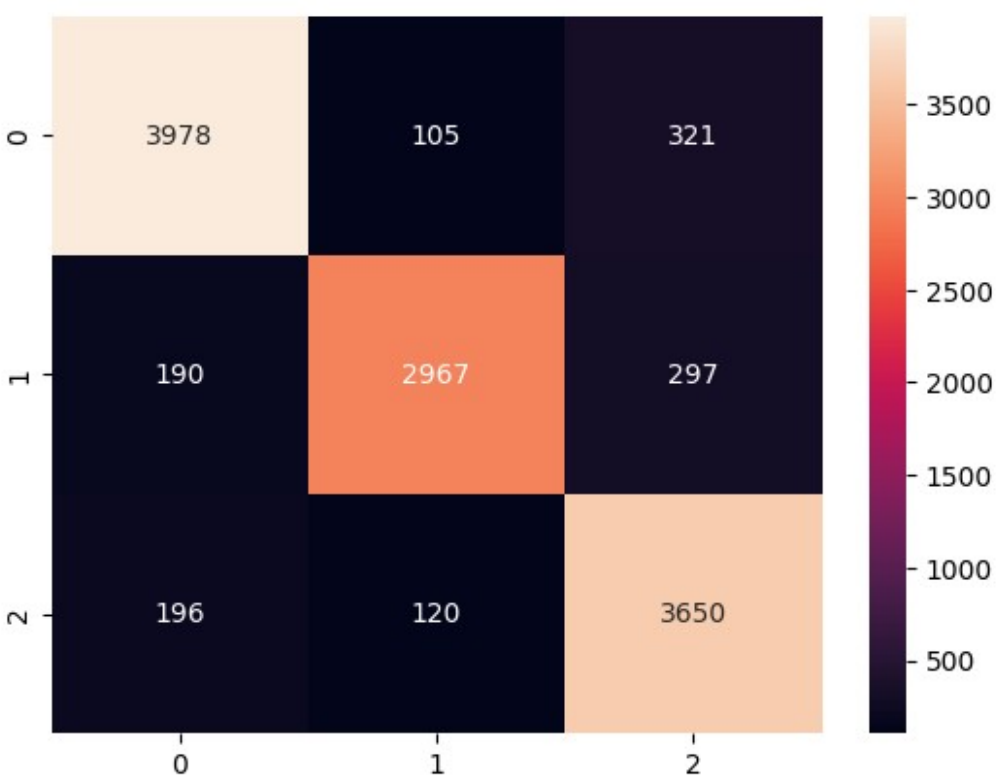
```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

```
RandomForestClassifier()
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.8960588633288228
              precision    recall  f1-score   support

           0       0.91      0.90      0.91      4404
           1       0.93      0.86      0.89      3454
           2       0.86      0.92      0.89      3966

    accuracy                           0.90     11824
   macro avg       0.90      0.89      0.90     11824
weighted avg       0.90      0.90      0.90     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.show()
```



#Gradient Boosting

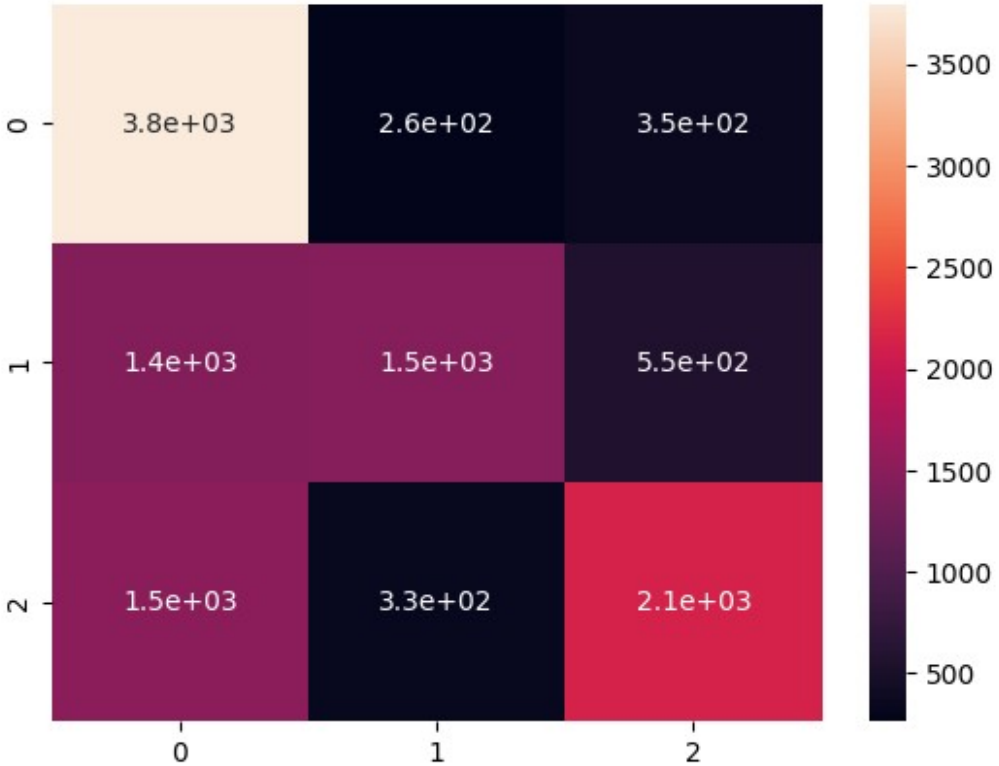
```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report

gb_model = GradientBoostingClassifier(n_estimators=100)
gb_model.fit(X_train, y_train)
```

```
GradientBoostingClassifier()
```

```
y_pred_gb = gb_model.predict(X_test)

print("Gradient Boosting Accuracy:", accuracy_score(y_test,
y_pred_gb))
print(classification_report(y_test, y_pred_gb))
```

```
Gradient Boosting Accuracy: 0.625169147496617
              precision    recall  f1-score   support

           0       0.56      0.86      0.68      4404
           1       0.71      0.42      0.53      3454
           2       0.70      0.54      0.61      3966

    accuracy                           0.63     11824
   macro avg       0.66      0.61      0.61     11824
weighted avg       0.65      0.63      0.61     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_gb), annot=True)
plt.show()
```



#AdaBoost

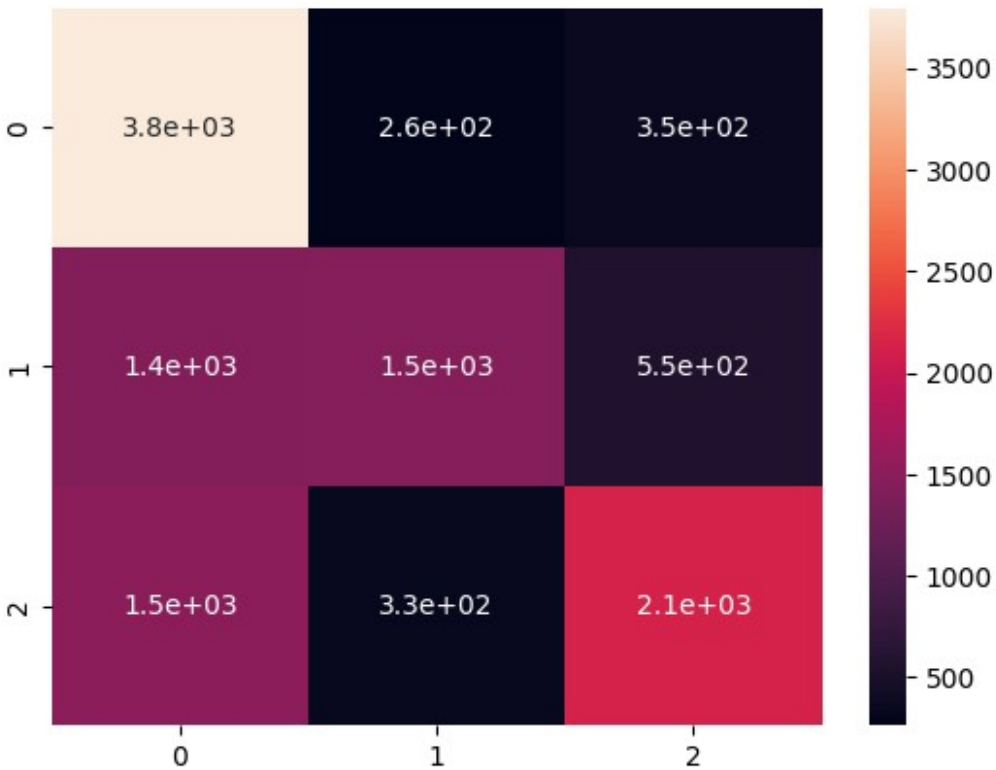
```
from sklearn.ensemble import AdaBoostClassifier

ada_model = AdaBoostClassifier(n_estimators=100)
ada_model.fit(X_train, y_train)
```

```
AdaBoostClassifier(n_estimators=100)
```

```
y_pred_ada = ada_model.predict(X_test)

print("AdaBoost Accuracy:", accuracy_score(y_test, y_pred_ada))
print(classification_report(y_test, y_pred_ada))
```

```
AdaBoost Accuracy: 0.515138700947226
              precision    recall  f1-score   support

           0       0.47      0.87      0.61      4404
           1       0.55      0.26      0.35      3454
           2       0.68      0.35      0.46      3966

    accuracy                           0.52     11824
   macro avg       0.56      0.49      0.47     11824
weighted avg       0.56      0.52      0.48     11824
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_ada), annot=True)
plt.show()
```



#XGBoost

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=100,
    max_depth=6,
    learning_rate=0.1,
    objective='multi:softmax',
    num_class=3,
    eval_metric='mlogloss'
)

xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)

print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

```
XGBoost Accuracy: 0.6578991880920162
              precision    recall  f1-score   support
```

```
           0       0.60      0.87      0.71      4404
           1       0.75      0.46      0.57      3454
           2       0.73      0.59      0.65      3966

    accuracy                           0.66     11824
   macro avg       0.69      0.64      0.64     11824
weighted avg       0.68      0.66      0.65     11824
```

```
y_pred_xgb = xgb_model.predict(X_test)

print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

```
XGBoost Accuracy: 0.6578991880920162
              precision    recall  f1-score   support

           0       0.60      0.87      0.71      4404
           1       0.75      0.46      0.57      3454
           2       0.73      0.59      0.65      3966

    accuracy                           0.66     11824
   macro avg       0.69      0.64      0.64     11824
weighted avg       0.68      0.66      0.65     11824
```

#BERT

```
!pip install transformers datasets torch
```

```
Requirement already satisfied: transformers in
/usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in
/usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: torch in
/usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (1.10.1)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from transformers)
(2025.11.3)
```

```
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer-slim in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
```

```
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4-
>torch) (1.5.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.13.5)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (1.4.3)
Requirement already satisfied: httpx<1,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (0.28.1)
Requirement already satisfied: typer in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.3.0->transformers) (0.24.1)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (2026.2.25)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets)
(2026.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.23.0)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.3.0->transformers) (4.13.0)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (0.16.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Requirement already satisfied: click>=8.2.1 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (8.3.2)
Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (1.5.4)
```

```
Requirement already satisfied: rich>=12.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer->huggingface-
hub<2.0,>=1.3.0->transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
>huggingface-hub<2.0,>=1.3.0->transformers) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
>huggingface-hub<2.0,>=1.3.0->transformers) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
(0.1.2)
```

```python
import torch
from transformers import BertTokenizer, BertForSequenceClassification,
Trainer, TrainingArguments
from datasets import Dataset
import numpy as np
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support

df = df[['clean_text', 'sentiment']]

# Convert to HuggingFace Dataset
dataset = Dataset.from_pandas(df)

# Train-test split
dataset = dataset.train_test_split(test_size=0.2)
train_dataset = dataset['train']
test_dataset = dataset['test']

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def tokenize(example):
    return tokenizer(
        example['clean_text'],
        padding='max_length',
        truncation=True,
        max_length=128
    )

train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)

# Rename the 'sentiment' column to 'labels' as expected by
BertForSequenceClassification
```

```
train_dataset = train_dataset.rename_column('sentiment', 'labels')
test_dataset = test_dataset.rename_column('sentiment', 'labels')

# Set format for PyTorch
train_dataset.set_format(type='torch', columns=['input_ids',
'attention_mask', 'labels'])
test_dataset.set_format(type='torch', columns=['input_ids',
'attention_mask', 'labels'])
```

{"model_id":"d7662bd816dc4f1fbdbce11c4cfca54b","version_major":2,"version_minor":0}

{"model_id":"ec404a56746b48ee87229def0d42bc15","version_major":2,"version_minor":0}

```
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=3
)
```

{"model_id":"a653116566d243979353908f03766f41","version_major":2,"version_minor":0}

{"model_id":"8e0fe6cf20a94108a03254d7d94de681","version_major":2,"version_minor":0}

{"model_id":"31b5da8a90ad42378f30eeca44f30ef1","version_major":2,"version_minor":0}

```
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
classifier.weight                            | MISSING    |
classifier.bias                              | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
- MISSING  :those params were newly initialized because missing from
the checkpoint. Consider training on your downstream task.
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
```

```
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
preds, average='weighted')
    acc = accuracy_score(labels, preds)

    return {
        'accuracy': acc,
        'f1': f1,
        'precision': precision,
        'recall': recall
    }

training_args = TrainingArguments(
    output_dir='./results',
    eval_strategy='epoch',
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir='./logs'
)
```

`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

```
<IPython.core.display.HTML object>
```


```
{"model_id":"16a17d6f052f4d3795b23380e9bf7ab3","version_major":2,"vers
ion_minor":0}

{"model_id":"c829c3eb6af1461cbbd3a16b1f5f2e25","version_major":2,"vers
ion_minor":0}

{"model_id":"9f8a32f453ee476986b27af934112069","version_major":2,"vers
ion_minor":0}

{"model_id":"364eef97f05340deab2322947fdd16cd","version_major":2,"vers
ion_minor":0}

{"model_id":"af97b2620d674ebab4d1cf5ba89bdbcb","version_major":2,"vers
ion_minor":0}
```

{"model_id":"5edf3e8c1ee54b6485304f2baeee55f1","version_major":2,"version_minor":0}

{"model_id":"79289b0dbe644f5c95b8a3ed01fbbf23","version_major":2,"version_minor":0}

{"model_id":"93bec7c5cf1549b1ad4a75199168792a","version_major":2,"version_minor":0}

{"model_id":"1e6859af419e49b49394da93b55f3e3c","version_major":2,"version_minor":0}

{"model_id":"96cc73e2ca5c4629b48b4231e1f43b2a","version_major":2,"version_minor":0}

{"model_id":"b352c6e0b99c4592b47b2830ac1d279f","version_major":2,"version_minor":0}

{"model_id":"0102c6d3da6a489e8cfdcce147de36e6","version_major":2,"version_minor":0}

{"model_id":"c0a3041daf9b4427944759f1d9c4ce5e","version_major":2,"version_minor":0}

{"model_id":"fdd9fbf758764fb29e6bf273b2ffae48","version_major":2,"version_minor":0}

{"model_id":"46f7383f32d14b73bc011135d0d5e447","version_major":2,"version_minor":0}

{"model_id":"63adb317c1d944f8a65db4452bf2a59b","version_major":2,"version_minor":0}

{"model_id":"f56e86c9594944d9a1a46ab77a8f13ad","version_major":2,"version_minor":0}

{"model_id":"04351ded798045d1aa3b0adefdfdf2b3","version_major":2,"version_minor":0}

{"model_id":"46f23c685c8c4efeaca4ab3b613681b3","version_major":2,"version_minor":0}

{"model_id":"ce660468a7854c77b0da92cb673ed4fc","version_major":2,"version_minor":0}

{"model_id":"48e2833e61fa4653a503f6104e125743","version_major":2,"version_minor":0}

{"model_id":"9b89e338b3bd42b88943897de6d6f222","version_major":2,"version_minor":0}

{"model_id":"62953e855644437f9cdde1fdc294e23c","version_major":2,"version_minor":0}

```
TrainOutput(global_step=11498, training_loss=0.48221503865057375,
metrics={'train_runtime': 3079.2186, 'train_samples_per_second':
29.87, 'train_steps_per_second': 3.734, 'total_flos':
6050029927237632.0, 'train_loss': 0.48221503865057375, 'epoch': 2.0})
```

```
trainer.evaluate()
```

```
<IPython.core.display.HTML object>
```

```
{'eval_loss': 0.42648202180862427,
 'eval_accuracy': 0.8907541097677655,
 'eval_f1': 0.8907508152215777,
 'eval_precision': 0.8907478617021366,
 'eval_recall': 0.8907541097677655,
 'eval_runtime': 95.3906,
 'eval_samples_per_second': 120.526,
 'eval_steps_per_second': 15.075,
 'epoch': 2.0}
```

```
trainer.save_model("bert_sentiment_model")
tokenizer.save_pretrained("bert_sentiment_model")
```


```
{"model_id":"f4b93e379c6a4027a3c580c8b91f0c9f","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"eafdb948d60c46e1ae47ecc3da191977","version_major":2,"vers
ion_minor":0}
```


```
('/content/drive/MyDrive/3rd year
labs/CI/TwitterSentimentAnalysis/bert_sentiment_model/tokenizer_config
.json',
 '/content/drive/MyDrive/3rd year
labs/CI/TwitterSentimentAnalysis/bert_sentiment_model/tokenizer.json')
```

```
preds = trainer.predict(test_dataset)
y_pred = np.argmax(preds.predictions, axis=1)
```

```
<IPython.core.display.HTML object>
```

#KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Initialize model
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

# Train
```

```
knn.fit(X_train, y_train)

# Predict
y_pred_knn = knn.predict(X_test)

# Evaluate
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```
KNN Accuracy: 0.883372801082544
              precision    recall  f1-score   support

           0       0.90      0.89      0.90      4404
           1       0.84      0.90      0.87      3454
           2       0.91      0.85      0.88      3966

    accuracy                           0.88     11824
   macro avg       0.88      0.88      0.88     11824
weighted avg       0.89      0.88      0.88     11824
```

#Perceptron

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report

# Initialize model
perceptron = Perceptron(max_iter=1000, random_state=42)

# Train
perceptron.fit(X_train, y_train)

# Predict
y_pred_perc = perceptron.predict(X_test)

# Evaluate
print("Perceptron Accuracy:", accuracy_score(y_test, y_pred_perc))
print(classification_report(y_test, y_pred_perc))
```

```
Perceptron Accuracy: 0.7281799729364006
              precision    recall  f1-score   support

           0       0.77      0.78      0.77      4404
           1       0.65      0.71      0.68      3454
           2       0.76      0.69      0.72      3966

    accuracy                           0.73     11824
   macro avg       0.73      0.73      0.73     11824
weighted avg       0.73      0.73      0.73     11824
```

#Nueral Network

Data cleaning and splitting

```
train_df = pd.read_csv("twitter_training.csv", header=None) #upadate
if needed
valid_df = pd.read_csv("twitter_validation.csv", header=None) #upadate
if needed

train_df.columns = ['id', 'entity', 'sentiment', 'text']
valid_df.columns = ['id', 'entity', 'sentiment', 'text']

# Remove nulls
train_df = train_df.dropna(subset=['text'])
valid_df = valid_df.dropna(subset=['text'])

# Remove irrelevant
train_df = train_df[train_df['sentiment'] != 'Irrelevant']
valid_df = valid_df[valid_df['sentiment'] != 'Irrelevant']

# Remove duplicates
train_df = train_df.drop_duplicates()
valid_df = valid_df.drop_duplicates()

stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return ' '.join(words)

train_df['text'] = train_df['text'].apply(clean_text)
valid_df['text'] = valid_df['text'].apply(clean_text)

enc = LabelEncoder()

train_df['sentiment'] = enc.fit_transform(train_df['sentiment'])
valid_df['sentiment'] = enc.transform(valid_df['sentiment'])

# Save mapping
label_mapping = dict(zip(enc.classes_, enc.transform(enc.classes_)))
print(label_mapping)
```

```
{'Negative': np.int64(0), 'Neutral': np.int64(1), 'Positive':
np.int64(2)}
```

```
X_train_text, X_test_text, y_train, y_test = train_test_split(
    train_df['text'],
```

```
    train_df['sentiment'],
    test_size=0.1,
    random_state=42
)

tfidf = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1,2),
    min_df=2,
    max_df=0.9
)

# Fit ONLY on train
X_train = tfidf.fit_transform(X_train_text)

# Transform others
X_test = tfidf.transform(X_test_text)
X_val = tfidf.transform(valid_df['text'])
y_val = valid_df['sentiment']
```

##Nueral Network

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Ensure numpy arrays
X_train_nn = X_train.toarray()
X_val_nn = X_val.toarray()
X_test_nn = X_test.toarray()

y_train_nn = y_train
y_val_nn = y_val
y_test_nn = y_test

model = Sequential()

# Input layer
model.add(Dense(512, activation='relu',
input_shape=(X_train_nn.shape[1],)))
model.add(Dropout(0.3))

# Hidden layer
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.3))

# Another hidden layer
model.add(Dense(128, activation='relu'))
```

```
# Output layer (3 classes)
model.add(Dense(3, activation='softmax'))

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential"
```

```
┏━━━
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=2,
    restore_best_weights=True
)

history = model.fit(
    X_train_nn, y_train_nn,
    validation_data=(X_val_nn, y_val_nn),
    epochs=10,
    batch_size=32,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 120s 71ms/step - accuracy: 0.7766 -
loss: 0.5368 - val_accuracy: 0.9517 - val_loss: 0.1497
Epoch 2/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 123s 74ms/step - accuracy: 0.9291 -
loss: 0.1738 - val_accuracy: 0.9650 - val_loss: 0.1231
Epoch 3/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 142s 74ms/step - accuracy: 0.9543 -
loss: 0.1006 - val_accuracy: 0.9662 - val_loss: 0.1225
Epoch 4/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 141s 73ms/step - accuracy: 0.9601 -
loss: 0.0837 - val_accuracy: 0.9674 - val_loss: 0.1125
Epoch 5/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 120s 72ms/step - accuracy: 0.9620 -
loss: 0.0776 - val_accuracy: 0.9674 - val_loss: 0.1589
Epoch 6/10
1663/1663 ━━━━━━━━━━━━━━━━━━━━ 118s 71ms/step - accuracy: 0.9638 -
loss: 0.0727 - val_accuracy: 0.9638 - val_loss: 0.1602
```

```
test_loss, test_acc = model.evaluate(X_test_nn, y_test_nn)

print("Test Accuracy:", test_acc)
```

```
185/185 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.9110 - loss:
0.2654
Test Accuracy: 0.9110284447669983
```

```
import numpy as np
from sklearn.metrics import classification_report

y_pred_nn = np.argmax(model.predict(X_test_nn), axis=1)
```

```
print(classification_report(y_test_nn, y_pred_nn))
```

```
185/185 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step
              precision    recall  f1-score   support

           0       0.87      0.95      0.91      2195
           1       0.93      0.88      0.91      1729
           2       0.94      0.89      0.92      1988

    accuracy                           0.91      5912
   macro avg       0.92      0.91      0.91      5912
weighted avg       0.91      0.91      0.91      5912
```

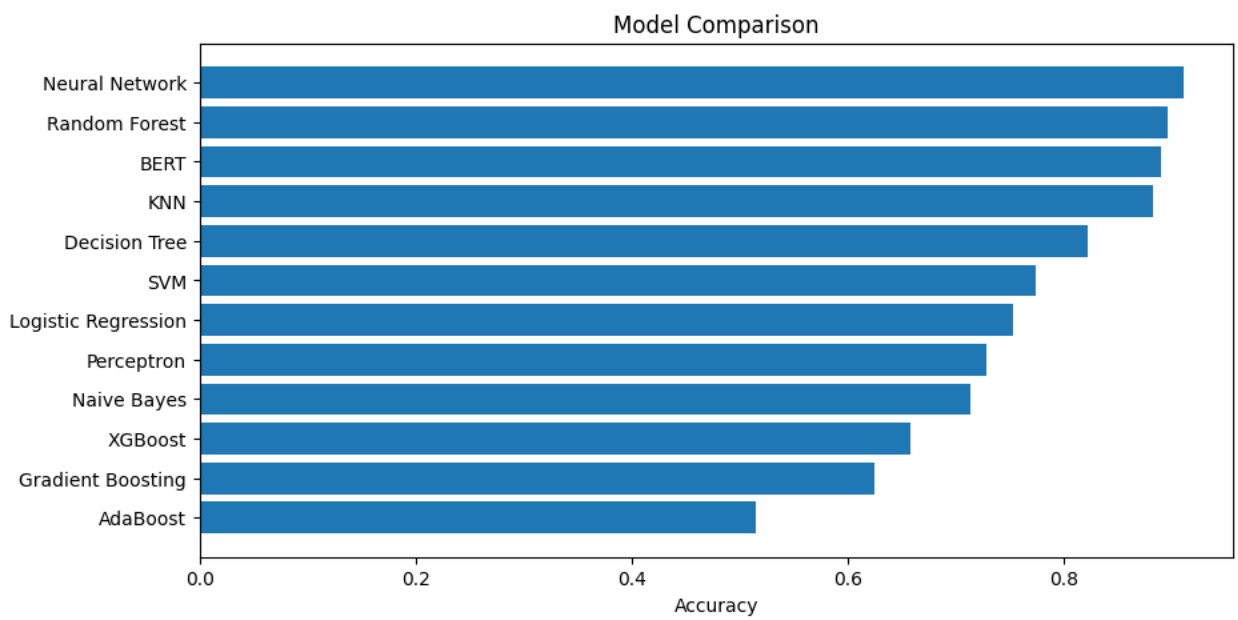
##Final Comparison of all models

```
results = {
    "Logistic Regression": 0.7537212449255751,
    "Naive Bayes": 0.7133795669824087,
    "SVM": 0.7744418132611637,
    "Decision Tree": 0.8219722598105548,
    "Random Forest": 0.8960588633288228,
    "KNN": 0.883372801082544,
    "Perceptron": 0.7281799729364006,
    "Gradient Boosting": 0.625169147496617,
    "AdaBoost": 0.515138700947226,
    "XGBoost": 0.6578991880920162,
    "Neural Network": 0.9110284447669983,
    "BERT": 0.8907541097677655
}

import pandas as pd

df_results = pd.DataFrame(list(results.items()), columns=["Model",
"Accuracy"])
df_results = df_results.sort_values(by="Accuracy", ascending=False)

print(df_results)
```

```
                   Model  Accuracy
10        Neural Network  0.911028
4          Random Forest  0.896059
11                  BERT  0.890754
5                    KNN  0.883373
3          Decision Tree  0.821972
2                    SVM  0.774442
0   Logistic Regression  0.753721
6             Perceptron  0.728180
1            Naive Bayes  0.713380
```

```
9                XGBoost  0.657899
7     Gradient Boosting  0.625169
8               AdaBoost  0.515139
```

```
best_model = max(results, key=results.get)
best_accuracy = results[best_model]

print(f"Best Model: {best_model}")
print(f"Best Accuracy: {best_accuracy}")
```

```
Best Model: Neural Network
Best Accuracy: 0.9110284447669983
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.barh(df_results["Model"], df_results["Accuracy"])
plt.xlabel("Accuracy")
plt.title("Model Comparison")
plt.gca().invert_yaxis()
plt.show()
```